Inteligencia Artificial · Tareas 1, 2 y 3

Estado del arte, agente aspiradora y reglas de navegación

Consulta de fuentes: 3 de noviembre de 2026. Enunciados: carpeta presentaciones.

Tarea 1. Agentes, asistentes y entidades inteligentes

Se analizan doce sistemas: diez asistentes o entidades de uso e investigación actuales, más ACT-R y NARS para conectar la investigación con los antecedentes de la presentación 3. Se describe la arquitectura pública de cada sistema y se distingue la aplicación del modelo que utiliza: un asistente puede cambiar de modelo, herramientas o permisos. Cuando la topología neuronal es propietaria y no está publicada en las fuentes revisadas, se indica expresamente. [P3]

AGI es la sigla habitual de inteligencia artificial general; corresponde a la GAI de la consigna. Para valorar avances se consideran amplitud de tareas, desempeño y autonomía, dimensiones del marco de Morris y colaboradores. Las valoraciones siguientes son análisis propio de capacidades documentadas. Satisfacer objetivos parciales —por ejemplo, transferir conocimientos entre tareas— no demuestra haber alcanzado competencia general humana. [G1]

1. ChatGPT, con funciones de agente en Work (OpenAI)

Arquitectura: interfaz de lenguaje natural sobre modelos seleccionables, contexto de conversación y archivos, planificación y orquestación de herramientas. El trabajo puede ejecutarse en un entorno local o en la nube. Las fuentes consultadas no detallan la topología neuronal, el número de capas o los parámetros de todos los modelos vigentes; sería incorrecto inventar esos datos. Experticia documentada: redacción, investigación, análisis de archivos, elaboración de documentos, programación y coordinación de tareas. Software/herramientas: búsqueda web, navegador, ejecución de código y comandos, plugins, skills, archivos y subagentes, según la modalidad y los permisos.

Relación con AGI: satisface parcialmente objetivos de versatilidad y transferencia entre tareas; esas capacidades no prueban por sí solas inteligencia general humana ni autonomía universal. Turing: no encontré una tasa comparable para el producto y su configuración actual en estas fuentes; no corresponde trasladarle el porcentaje de GPT-4.5 de un experimento histórico. La fluidez y el manejo de contexto justifican estudiarlo, pero no permiten calcular una probabilidad individual. [A1–A2]

2. Claude (Anthropic)

Arquitectura: familia de modelos de lenguaje con entrada de texto e imágenes, salida textual y uso de herramientas; puede integrarse en un ciclo agente–herramienta–resultado. La descripción pública consultada no publica una arquitectura neuronal completa, así que no se afirma un número de parámetros ni que todas sus versiones sean MoE. Experticia: razonamiento, programación, trabajo documental, clasificación, extracción de información y tareas de varios pasos. Software/herramientas: Messages API; herramientas de capturas de pantalla, ratón y teclado; Bash y editor de texto que pueden acompañar al uso del ordenador. En las herramientas de cliente es la aplicación quien ejecuta las acciones y devuelve sus resultados.

Relación con AGI: aporta flexibilidad lingüística, visión y acción digital, pero esto no acredita desempeño general humano. Turing: porcentaje no verificado para la versión actual en estas fuentes; la expresión natural favorece semejanza conversacional, mientras que errores, contexto y restricciones de identidad pueden volverla detectable. [A3–A4]

3. Gemini (Google DeepMind)

Arquitectura: familia multimodal con herramientas. Como referencia técnica pública concreta, el informe de Gemini 2.5 de 2025 describe Transformers de mezcla dispersa de expertos (MoE), con enrutamiento que activa sólo parte de los expertos por token, y entradas de texto, visión y audio. Este dato corresponde a 2.5 y no demuestra que cada versión posterior tenga idéntica arquitectura. Experticia: comprensión multimodal, razonamiento, programación, matemáticas y análisis de información. Software/herramientas: Gemini API; Google Search y Maps, URL Context, File Search/RAG, ejecución de Python, function calling y uso de interfaces de navegador, según modelo y configuración.

Relación con AGI: combina modalidades y tareas distintas, objetivos parciales de generalidad; no equivale a demostrar AGI. Turing: porcentaje no establecido en las fuentes consultadas; su calidad conversacional podría hacerlo plausible, pero las pruebas de otras familias y las evaluaciones de preferencia no miden su tasa en una prueba de Turing. [A5–A6]

4. Microsoft Copilot, con integración Microsoft 365

Arquitectura: LLMs más recuperación o grounding sobre la web y/o información organizacional, control de permisos e integración con aplicaciones. En experiencias habilitadas, Microsoft Graph y Work IQ aportan contexto de archivos, correos y reuniones. No es un único modelo con una sola arquitectura neuronal. Experticia: productividad de oficina, redacción, análisis de hojas de cálculo, preparación de presentaciones, resúmenes de correos y reuniones. Software/herramientas: Word, Excel, PowerPoint, Outlook, Teams, OneNote, Microsoft Graph, Work IQ, búsqueda y agentes disponibles según licencia.

Relación con AGI: generaliza entre tareas de oficina; su ámbito y sus conectores siguen delimitados, por lo que eso no acredita AGI. Turing: no hay una tasa verificable en la documentación revisada; podría producir conversaciones fluidas gracias a sus modelos, pero el conocimiento empresarial y la calidad de una presentación no son evidencia de indistinguibilidad humana. [A7]

5. GitHub Copilot

Arquitectura: plataforma de asistencia y agentes con modelos y herramientas de distintos proveedores, contexto de repositorio y sesiones. La documentación vigente distingue cliente, sesión y entorno de ejecución; éste puede ser un sandbox en la nube, un runner de GitHub Actions u otro entorno. No debe describirse exclusivamente con su arquitectura antigua. Experticia: escribir y explicar código, investigar repositorios, planificar cambios, ejecutar pruebas y linters, revisar código y preparar pull requests. Software/herramientas: GitHub, VS Code y CLI, edición de archivos, búsqueda semántica de código, entornos de desarrollo, GitHub Actions y servidores MCP.

Relación con AGI: muestra planificación y ejecución autónoma parcial dentro del desarrollo de software; la especialización y los modelos subyacentes no bastan para acreditarlo como AGI. Turing: porcentaje no verificado; resolver programación no es la prueba de conversación general. Un ensayo tendría que fijar versión, interfaz y protocolo. [A8–A9]

6. Perplexity, incluido su modo de investigación y Computer

Arquitectura: combina modelos de lenguaje, recuperación de información web y síntesis con citas. Su arquitectura pública Search as Code (SaC) añade tres capas: modelos que deciden y generan el plan de búsqueda, un entorno aislado que ejecuta el código y un Agentic Search SDK que expone primitivas de recuperación. Esto describe la orquestación; no revela una arquitectura neuronal única ni todos los parámetros de los modelos seleccionables. Se especializa en investigación documental, consultas con fuentes y análisis. Herramientas documentadas: búsqueda web, intérprete de código y, en SaC, código Python, entornos aislados y Agentic Search SDK. La documentación de Pro Search permite seleccionar distintos modelos, por lo que no se debe atribuir a todo Perplexity una sola red.

Valoración sobre AGI: integra lenguaje, planificación de búsquedas y uso de herramientas en varios temas, objetivos parciales de generalidad; esto no establece inteligencia general de nivel humano. Turing: no localicé un porcentaje experimental verificable para el producto actual. La fluidez conversacional permite plantear esa prueba, pero sus citas y estilo de asistente, así como el modelo y las instrucciones escogidas, alterarían el resultado. No se heredan porcentajes de los modelos externos. [B1, B2]

7. Alexa+ (Amazon)

Arquitectura: un sistema de orquestación conecta varios modelos de lenguaje, servicios y dispositivos. Un enrutador selecciona modelos mediante Amazon Bedrock, incluidos Amazon Nova y Anthropic Claude; la arquitectura encadena llamadas a API y contempla navegación de sitios para completar tareas. No se publican en estas fuentes el número de parámetros ni la arquitectura neuronal completa de cada componente. Sus áreas de especialidad son asistencia por voz, hogar conectado, entretenimiento, reservas y organización personal. Software documentado: Amazon Bedrock, las API de servicios y Alexa AI Action SDK, Alexa AI Web Action SDK y Alexa AI Multi-Agent SDK; estos SDK son interfaces de integración, no lenguajes o marcos de entrenamiento inferidos.

Valoración sobre AGI: combina conversación, personalización y ejecución de tareas en diferentes servicios, capacidades parciales de autonomía; su alcance sigue dependiendo de integraciones, permisos y de los modelos disponibles. Turing: no localicé un porcentaje experimental publicado y verificable para Alexa+ en la prueba conversacional humana. Podría evaluarse, pero el idioma, la modalidad de voz o texto y el modo de identificación del asistente cambian el protocolo; no hay base para asignarle una probabilidad numérica confiable. [B3, B4]

8. Siri AI con Apple Intelligence (Apple, generación 2026)

Arquitectura: combina modelos locales y modelos en servidores mediante Private Cloud Compute. La familia AFM de tercera generación incluye AFM 3 Core, un modelo denso de aproximadamente 3 mil millones de parámetros, y AFM 3 Core Advanced, un Transformer de activación dispersa; AFM 3 Cloud utiliza una base Parallel-Track Mixture-of-Experts. Sus especialidades son asistencia personal, conversación, consultas sobre contenido en pantalla, búsqueda de información personal y acciones en aplicaciones. Software documentado: orquestador del sistema, índice Spotlight, App Toolbox y Private Cloud Compute.

Valoración sobre AGI: combina lenguaje, percepción visual, contexto y acción en distintos programas, avances parciales de generalidad; sus funciones dependen del ecosistema y del acceso. Turing: no localicé un porcentaje medido y verificable para Siri AI; la voz natural por sí sola no prueba indistinguibilidad ni AGI. Importante sobre disponibilidad: el comunicado del 14 de septiembre de 2026 confirma el despliegue beta en inglés, contexto personal y acciones en pantalla; no deben describirse globalmente como meras promesas de 2024. Ese comunicado anunciaba otros idiomas para el mes siguiente y restricciones regionales. No corroboré el despliegue efectivo completo posterior de esos idiomas; el anuncio no lo demuestra. [B5, B6]

9. AlphaFold 3 (Google DeepMind e Isomorphic Labs)

Arquitectura: una red profunda procesa representaciones de las moléculas y sus relaciones; un modelo de difusión genera y refina coordenadas tridimensionales. El sistema aborda conjuntamente complejos que pueden incluir proteínas, ADN, ARN y ligandos. Sus áreas de especialidad son predicción estructural biomolecular e interacciones moleculares, con utilidad para investigación biológica y diseño de fármacos; estas predicciones no sustituyen la comprobación experimental. Software documentado en el repositorio oficial de inferencia: Python, JAX, Haiku, NumPy, RDKit, HMMER y Docker. El código público es de inferencia, no equivale a publicar todo el proceso de entrenamiento.

Valoración sobre AGI: muestra aprendizaje y generalización dentro de un dominio científico amplio, pero permanece especializado; no demuestra competencia general en lenguaje, vida cotidiana o múltiples tareas humanas. Turing: no aplica directamente una probabilidad de aprobar la prueba conversacional, porque su interfaz produce estructuras y no pretende mantener un diálogo humano general. “No aplica” no significa 0 %; añadirle un chatbot evaluaría también otro sistema. [B7, B8]

10. Waymo Driver (Waymo)

Arquitectura: sistema de conducción autónoma con percepción, predicción de otros usuarios de la vía y planificación de trayectorias y acciones. Integra información de cámaras, lidar, radar y mapas detallados mediante aprendizaje automático y cómputo a bordo. La descripción pública de sus módulos no equivale a revelar todas sus redes, parámetros o código de producción. Su especialidad es conducción autónoma en los entornos operativos para los que se valida. Software documentado: sistemas propios de percepción, predicción, planificación y control, y SimulationCity para generar escenarios, entrenar modelos y evaluar nuevas versiones. No atribuyo a la producción un lenguaje o framework concreto no confirmado.

Valoración sobre AGI: realiza percepción y acción autónoma en un ambiente físico complejo, objetivos relevantes para agentes inteligentes; su competencia sigue siendo específica de la conducción, no general en todos los dominios. Turing: no aplica directamente un porcentaje de aprobación conversacional al Driver; para compararlo con humanos corresponde estudiar conducción y seguridad, no suplantación en un diálogo. Un eventual asistente de voz del vehículo sería una entidad distinta. [B9, B10]

11. ACT-R

Es una arquitectura cognitiva híbrida: representa conocimiento declarativo mediante unidades simbólicas y habilidades mediante reglas de producción; parámetros subsimbólicos regulan activación, recuperación y selección. Integra módulos de memoria, metas y percepción–acción mediante buffers. Su experticia es modelar memoria, aprendizaje, atención, razonamiento e interacción humano–computadora, comparando tiempos y errores con datos humanos. Su implementación canónica utiliza Common Lisp y herramientas del entorno ACT-R.

Como objetivo relacionado con AGI, busca integrar procesos cognitivos en un marco general; eso no demuestra que un modelo específico posea inteligencia general humana. Continúa investigándose: un trabajo de 2026 integra memoria vectorial y fuentes de conocimiento con ACT-R. Turing: no identifiqué una tasa verificada; sus modelos se evalúan sobre tareas cognitivas y no constituyen, por defecto, un interlocutor conversacional general. [C1–C2]

12. NARS / OpenNARS

NARS es un sistema de razonamiento adaptativo con memoria, motor de inferencia y control de recursos. Utiliza lógica no axiomática (NAL) y el lenguaje de representación Narsese; sus creencias pueden revisarse cuando cambia la evidencia. Su especialidad es inferir, aprender, predecir y perseguir metas con conocimiento y recursos insuficientes. La implementación OpenNARS for Research 3.x documenta Java/OpenJDK y Maven, no Prolog; existen otras implementaciones, por lo que no debe atribuirse un único lenguaje a toda la familia. A diferencia de un asistente limitado a una tarea, el proyecto declara explícitamente el objetivo de construir AGI, pero la intención del diseño no certifica haberla alcanzado. Turing: no identifiqué un porcentaje experimental verificable; razonar en Narsese tampoco equivale a sostener una conversación humana indistinguible. [C3–C4]

Turing: porcentajes publicados y su interpretación

El porcentaje siguiente es la fracción de partidas en que un juez eligió a la IA como el interlocutor humano, frente a una persona real. No representa “porcentaje de inteligencia” ni una probabilidad universal de aprobar. El estudio de Jones y Bergen empleó conversaciones por texto, tres participantes y configuraciones específicas; en la condición PERSONA se pidió imitar una personalidad humana. [T1]

Sistema y condición del estudio	Duración	Elegido como humano
GPT-4.5, PERSONA	5 minutos	73 %
Llama-3.1-405B, PERSONA	5 minutos	56 %
GPT-4o, sin PERSONA	5 minutos	21 %
ELIZA, referencia histórica	5 minutos	23 %
GPT-5, PERSONA	15 minutos	59.3 % (89/150 partidas)

Fuente: [T1], tablas 1–2. El 50 % es la referencia de elección al azar en este protocolo. Los autores concluyen que GPT-4.5, Llama-3.1 y GPT-5 cumplen su criterio bajo las condiciones estudiadas. El estilo conversacional y las instrucciones influyen en el resultado. Estos datos no pueden transferirse a ChatGPT, Claude, Gemini u otros productos con versiones y configuraciones diferentes. Para los doce sistemas descritos se conserva “sin porcentaje verificable” o “no aplica directamente”, según sus interfaces. No se inventan estimaciones numéricas.

La prueba examina semejanza conversacional; por sí sola no acredita AGI. Los asistentes de propósito amplio aportan versatilidad; Copilot se concentra en trabajo o programación; AlphaFold y Waymo demuestran capacidades especializadas; ACT-R y NARS exploran integración cognitiva y razonamiento. Las fuentes consultadas permiten reconocer avances parciales, pero no acreditar AGI completa en ninguno de estos sistemas. [G1; T1]

Tarea 2. Agente aspiradora de reflejo simple

La página 9 de la presentación 5 pide implementar el agente, simular todas las configuraciones iniciales y registrar puntajes y promedio. Se sigue ese enunciado, identificado allí como ejercicio 2.9; la numeración varía entre ediciones del libro. [P5]

Se utiliza el mundo de dos casillas A y B del libro: posición inicial desconocida para el diseñador del ensayo, suciedad inicial en cualquiera de las casillas, suciedad estática y acciones deterministas. El agente sólo percibe su posición y si esa casilla está sucia. El horizonte es de 1000 pasos y se otorga un punto por casilla limpia en cada paso, sin penalización de movimiento. Estos supuestos proceden del capítulo público del libro. [L1]

Hay 2 posiciones iniciales × 2 estados de A × 2 estados de B = 8 casos. La regla es: si la casilla actual está sucia, aspirar; si está limpia y el agente está en A, ir a B; si está limpia y está en B, ir a A. El programa del agente no guarda historial ni conoce directamente la suciedad de la otra casilla.

```
def agente_reflejo(posicion, sucio):
    if sucio:
        return 'aspirar'
    return 'derecha' if posicion == 'A' else 'izquierda'
```

Convención principal de puntuación: observar y puntuar el estado antes de ejecutar la acción de cada paso. El máximo es 2 × 1000 = 2000 puntos. Se calcula también la alternativa de puntuar después de actuar, porque ese desplazamiento temporal cambia los valores y debe quedar explícito. En ambas columnas se evalúan exactamente los mismos casos.

S = suma, para t = 0,...,999, de [1 si A está limpia en s_t] + [1 si B está limpia en s_t].

A inicial	B inicial	Inicio	Antes de actuar	Después de actuar
limpia	limpia	A	2000	2000
limpia	limpia	B	2000	2000
limpia	sucia	A	1998	1999
limpia	sucia	B	1999	2000
sucia	limpia	A	1999	2000
sucia	limpia	B	1998	1999
sucia	sucia	A	1996	1998
sucia	sucia	B	1996	1998
Promedio			1998.25	1999.25

Resultados de la ejecución de tarea2_aspiradora.py. Los ocho casos tienen el mismo peso para el promedio. No se ha supuesto una distribución probabilística distinta sobre los estados iniciales.

Ejemplo del puntaje mínimo: con A y B sucias e inicio en A, los primeros estados puntuados tienen 0, 1 y 1 casillas limpias. Las acciones son aspirar, derecha y aspirar; desde el estado siguiente hay dos casillas limpias. Por ello S = 0 + 1 + 1 + 997 × 2 = 1996. Si ambas estaban limpias, S = 2000. El programa deja las dos limpias en un máximo de tres acciones.

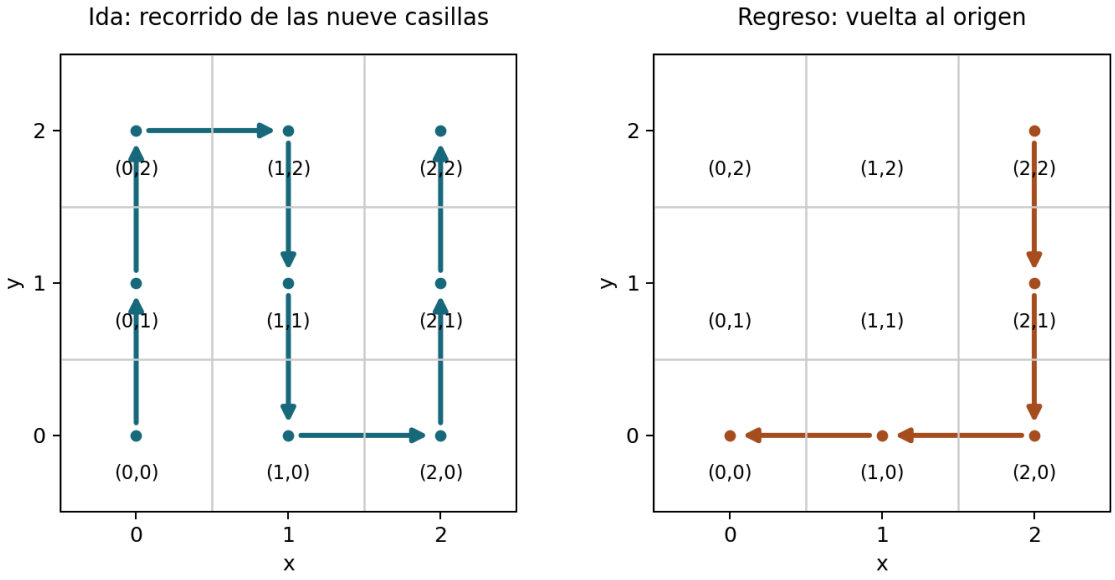
El archivo tarea2_aspiradora.py incluye el simulador completo, la enumeración de los ocho casos, los cálculos de ambas convenciones, la exportación CSV/JSON y las verificaciones de estos resultados.

Una vez limpio, el agente sigue alternando A y B: funciona para esta medida sin coste de movimiento, pero no sabe cuándo dejar de moverse. Una penalización energética cambiaría el problema y justificaría memoria para reconocer que ya limpió ambas casillas. Se resolvió el ejercicio solicitado, no la variante con penalización. [L1]

Tarea 3. Reglas complementarias de navegación

La presentación 6 define un cuarto de 3 × 3, inicio (0,0,norte) y tres acciones: avanzar una casilla, aspirar y girar 90° a la derecha. Se debe recorrer continuamente el cuarto. Los ejes son x al este e y al norte; los giros siguen N → E → S → O → N. [P6, pp. 10–13]

Ida requerida: (0,0) → (0,1) → (0,2) → (1,2) → (1,1) → (1,0) → (2,0) → (2,1) → (2,2). La diapositiva no fija el regreso; se propone (2,2) → (2,1) → (2,0) → (1,0) → (0,0), y un giro final para recuperar el norte. Todos los cambios de rumbo se hacen con giros físicos a la derecha.



Los giros no cambian la casilla. El ciclo termina al recuperar (0,0) mirando al norte.

Figura propia. Las flechas representan avances; los giros conservan la casilla. La ruta de regreso es una elección de esta solución.

Extensión declarada: se añade el bit interno fase = ida o regreso, ausente de los tres predicados iniciales. No es una acción física nueva. La actualización de estado cambia a regreso al llegar a (2,2), y a ida al recuperar (0,0,norte).

La distinción evita un conflicto real: en (1,0,oeste), durante la ida se debe girar para continuar a (2,0), pero en el regreso se debe avanzar hacia (0,0). Sin distinguir fases no se pueden asignar ambas decisiones a la misma percepción. Volver desde (0,1,sur) tampoco evita el problema conservando las reglas originales: en (0,1,norte) éstas fuerzan avanzar, y en (0,2,este) fuerzan salir al este antes de poder girar al sur. La solución es, por tanto, un agente con estado interno, y esta ampliación se hace explícita.

La regla 1.1 tiene prioridad absoluta: en(x,y) y basura(x,y) → Do(aspira). Aspirar conserva posición, rumbo y fase. Cada regla de movimiento incluye que la casilla actual esté limpia; se aplica también esa guarda a 1.5, tal como exige la nota de la presentación. Las letras N, E, S y O significan norte, este, sur y oeste.

Regla original	Posición	Rumbo	Acción si está limpia
1.2	(0,0)	N	avanzar
1.3	(0,1)	N	avanzar
1.4	(0,2)	N	girar
1.5	(0,2)	E	avanzar

Cada fila siguiente representa: en(x,y) y orientación(d) y no basura(x,y) y fase(f) → Do(acción). Así se escriben todas las reglas complementarias sin repetir innecesariamente los mismos predicados.

Regla	Fase	Posición	Orientación	Acción
1.6	ida	(1,2)	este	girar
1.7	ida	(1,2)	sur	avanzar
1.8	ida	(1,1)	sur	avanzar
1.9	ida	(1,0)	sur	girar
1.10	ida	(1,0)	oeste	girar
1.11	ida	(1,0)	norte	girar
1.12	ida	(1,0)	este	avanzar
1.13	ida	(2,0)	este	girar
1.14	ida	(2,0)	sur	girar
1.15	ida	(2,0)	oeste	girar
1.16	ida	(2,0)	norte	avanzar
1.17	ida	(2,1)	norte	avanzar
1.18	regreso	(2,2)	norte	girar
1.19	regreso	(2,2)	este	girar
1.20	regreso	(2,2)	sur	avanzar
1.21	regreso	(2,1)	sur	avanzar
1.22	regreso	(2,0)	sur	girar
1.23	regreso	(2,0)	oeste	avanzar
1.24	regreso	(1,0)	oeste	avanzar
1.25	regreso	(0,0)	oeste	girar

Ejemplo de la regla 1.24 completa: en(1,0) y orientación(oeste) y no basura(1,0) y fase(regreso) → Do(avanzar). La regla 1.10 usa esa misma posición y rumbo, pero fase(ida), y produce girar. En (1,0), pasar de sur a este requiere tres giros a la derecha; ocurre lo mismo al pasar de este a norte en (2,0).

Verificación ejecutada: tarea3_reglas.py comprueba las 2^9 = 512 distribuciones iniciales, dos ciclos cada una, con basura estática. En todas visita las nueve casillas, limpia, permanece dentro del cuarto y recupera (0,0,norte). Un ciclo limpio consume 12 avances + 12 giros = 24 acciones; con k casillas

inicialmente sucias, el primero consume 24 + k (33 si las nueve están sucias). El siguiente ciclo limpio consume 24. La ruta no se presenta como óptima. El agente puede seguir repitiendo ciclos; sólo la simulación se limita a dos.

Referencias y materiales consultados

Se consultaron las páginas y documentos enlazados el 3 de noviembre de 2026. “s.f.” indica que la página no muestra una fecha editorial estable; no se interpreta como una fecha de publicación. Las referencias P3, P5 y P6 son los materiales proporcionados para la clase. Los resultados de las tareas 2 y 3 proceden de los programas ejecutados incluidos en la entrega.

[P3] Material de clase, sin fecha. 3-Estado del Arte de la IA-e.ppt.

[P5] Material de clase, sin fecha. 5-Agentes inteligentes-1de2.pdf, página 9 (Tarea 2).

[P6] Material de clase, sin fecha. 6-Agentes inteligentes-2de2.pdf, páginas 10–13 (Tarea 3).

[A1] OpenAI. Use ChatGPT. Documentación oficial, s.f.
https://learn.chatgpt.com/docs/use-chatgpt

[A2] OpenAI. Models. Documentación oficial, s.f.
https://learn.chatgpt.com/docs/models

[A3] Anthropic. Models overview. Documentación oficial, s.f.
https://platform.claude.com/docs/en/models/overview

[A4] Anthropic. Computer use tool. Documentación oficial, s.f.
https://platform.claude.com/docs/en/agents-and-tools/tool-use/computer-use-tool

[A5] Gemini Team, Google DeepMind (2025). Gemini 2.5: Pushing the Frontier with Advanced Reasoning, Multimodality, Long Context, and Next Generation Agentic Capabilities, sección 2.1.
https://storage.googleapis.com/deepmind-media/gemini/gemini_v2_5_report.pdf

[A6] Google. Using tools with Gemini API. Documentación oficial, s.f.
https://aistudio.google.com/docs/tools

[A7] Microsoft. Microsoft Copilot overview. Documentación oficial, s.f.
https://learn.microsoft.com/en-us/microsoft-365/copilot/copilot-overview

[A8] GitHub. About GitHub Copilot. Documentación oficial, s.f.
https://docs.github.com/en/copilot/get-started/about-github-copilot

[A9] GitHub. GitHub Copilot on GitHub.com and GitHub Mobile. Incluye la experiencia disponible desde el 28 de octubre de 2026.
https://docs.github.com/en/copilot/concepts/copilot-surfaces/copilot-on-github

[B1] Perplexity. Rethinking Search as Code Generation. Publicación técnica oficial, s.f.
https://www.perplexity.ai/en-GB/hub/blog/rethinking-search-as-code-generation

[B2] Perplexity. What is Pro Search? Documentación oficial, s.f.
https://www.perplexity.ai/help-center/en/articles/10352903-what-is-pro-search

[B3] Amazon. How Amazon rebuilt Alexa with generative AI. Publicación técnica oficial, s.f.
https://www.aboutamazon.com/news/devices/new-alexa-tech-generative-artificial-intelligence

[B4] Yoshitake, M. (26 de febrero de 2025). Introducing AI-native SDKs for Alexa+. Amazon Developer.
https://developer.amazon.com/en-US/blogs/alexa/alexa-skills-kit/2025/02/new-alexa-announce-blog

[B5] Apple Machine Learning Research (8 de junio de 2026; actualización 9 de septiembre de 2026). Introducing the Third Generation of Apple’s Foundation Models.
https://machinelearning.apple.com/research/introducing-third-generation-of-apple-foundation-models

[B6] Apple (14 de septiembre de 2026). Siri AI, a profoundly more capable and personal assistant powered by the next generation of Apple Intelligence, is here. Apple Newsroom.
https://www.apple.com/newsroom/2026/09/siri-ai-a-profoundly-more-capable-and-personal-assistant-is-here/

[B7] Isomorphic Labs (8 de mayo de 2024; actualización 11 de noviembre de 2024). AlphaFold 3 predicts the structure and interactions of all of life’s molecules.
https://www.isomorphiclabs.com/articles/alphafold-3-predicts-the-structure-and-interactions-of-all-of-lifes-molecules

[B8] Google DeepMind. AlphaFold 3 inference pipeline. Repositorio oficial, s.f.
https://github.com/google-deepmind/alphafold3

[B9] Waymo. Waymo Driver: Self-Driving Car Technology for a Reliable Ride. Descripción técnica oficial, s.f.
https://waymo.com/waymo-driver/

[B10] The Waymo Team (6 de julio de 2021). Simulation City: Introducing Waymo’s most advanced simulation system yet for autonomous driving.
https://waymo.com/blog/2021/07/simulation-city/

[C1] Ritter, F. E., Kim, J. W. y Oury, J. D. (20 de junio de 2017). ACT-R: Frequently Asked Questions List. Penn State, secciones G1, G4 y G8.
https://acs.ist.psu.edu/projects/act-r-faq/act-r-faq.html

[C2] Dulam, S., Gay, J. R. y Dancy, C. L. (2026). New knowledge source pipelines for sociocultural representations in a cognitive architecture. Frontiers in Psychology, 17, 1874556.
https://doi.org/10.3389/fpsyg.2026.1874556

[C3] Wang, P. (agosto de 2026). A Logical Model of Intelligence — An introduction to NARS. Temple University.
https://cis.temple.edu/~pwang/NARS-Intro.html

[C4] OpenNARS. OpenNARS for Research 3.0+. Repositorio oficial, s.f.
https://github.com/opennars/opennars

[G1] Morris, M. R. y colaboradores (2023; revisión 2025). Levels of AGI for Operationalizing Progress on the Path to AGI. arXiv:2311.02462.
https://arxiv.org/abs/2311.02462

[T1] Jones, C. R. y Bergen, B. K. (19 de mayo de 2026). Large language models pass a standard three-party Turing test. PNAS, 123(21), e2524472123. Tablas 1 y 2.
https://doi.org/10.1073/pnas.2524472123

[L1] Russell, S. y Norvig, P. Artificial Intelligence: A Modern Approach. Capítulo 2, Intelligent Agents, página impresa 36 del capítulo público: supuestos del mundo aspiradora.
https://aima.eecs.berkeley.edu/4th-ed/pdfs/newchap02.pdf

Archivos de apoyo

tarea2_aspiradora.py reproduce los ocho casos y exporta sus puntajes. tarea3_reglas.py reproduce la verificación de las reglas; tarea3_verificacion.json conserva una traza de un ciclo limpio. La carpeta resultados incluye los datos de la tarea 2 y la figura del recorrido. Los dos programas usan únicamente la biblioteca estándar de Python; pueden ejecutarse con python tarea2_aspiradora.py y python tarea3_reglas.py.